

Language Model Forge: An Early-Stage Design Science Study to Simplify Language Model Customization

Yaman Aljnadi (Graduate Student)
Cletus Ngwerume (Graduate Student)
Sibonginkosi Nkashe (Graduate Student)
Jeffrey Wall (Faculty Advisor)



**Michigan
Technological
University**

Agenda



Design Motivation

Existing Designs

Kernel Design Theories

Methodology (Pure Technology Design Study)

Initial Results

Conclusions

Motivation



- AI and language models are rapidly growing in research and business applications. However, customizing AI systems requires data science and programming knowledge.
- Many business researchers and domain experts lack this technical workflow knowledge
- As a result, they cannot easily create domain-specific AI tools for their research.

Goal for this study

- LMForge simplifies AI customization by automating fine-tuning dataset creation workflows for non-technical researchers.

Existing Designs



Recent advances in programming libraries and software tools have made it easier to design AI workflows. Libraries such as Tensorflow and Pytorch provide widely used tools for training and deploying language models.

Similarly, the HuggingFace ecosystem provides a simple interface for accessing and managing language models and datasets. HuggingFace ecosystem provides a simple interface for accessing and managing language models and datasets. HuggingFace also offers tools such as

- Transformers: for working with pretrained models
- SFTTrainer: for fine-tuning language models

These tools have enabled researchers and developers to build powerful domain-specific AI systems.

Existing Designs



To make AI development easier for non-programmers, several graphical workflow tools have been developed.

Examples include LangFlow, n8n, and ChainForge, which allow users to visually design AI pipelines by connecting components such as models, prompts, and data sources.

These tools provide flexibility for building a wide range of AI applications. However, they still require users to understand AI workflow design, model configuration, and data preparation.

As a result, many business researchers and domain experts still face barriers when attempting to create domain-specific AI systems.

Kernel Design Theories



In design science research, theories are not always directly tested. Instead, kernel theories guide the design of the artifact and inform key design decisions.

LMForge's design is informed by three theories:

Cognitive Load Theory (Sweller, Ayres, & Kalyuga, 1998)

- Human working memory has limited capacity.
- Complex systems overwhelm users with too much information.
- LMForge reduces cognitive load by hiding technical AI workflow details behind guided interfaces.

Kernel Design Theories

Bounded Rationality Theory (Simon, 1955)

- People cannot evaluate all possible options in complex systems.
- Users often choose “good enough” solutions rather than optimal ones.
- LMForge embeds best-practice configurations to reduce decision complexity.

Technology Acceptance Model (Davis, 1989)

- Technology adoption depends on perceived usefulness and ease of use.
- LMForge uses simple linear interaction views to make AI workflows easier for non-technical users.

Methodology: Two Model Approach



Model 1 (Encoder): Classifies raw text chunks as either relevant or irrelevant.

- The encoder is the quality-control step in the pipeline.
- It removes non-core text before generation, which reduces noise and unnecessary computation.

Model 2 (Decoder): Generates 1-2 question-answer pairs from the relevant chunks and formats them as strict JSON.

- These two models operate completely in the backend, hiding the complex workflow from the user.

Initial Results: Two Model Approach



The Challenge: Single models (4-7B parameters) struggled to reliably filter text *and* format complex JSON.

The Solution: The two-model pipeline successfully automates dataset creation while remaining lightweight.

Methodology: Hardware Configuration



- **Workstation:** Dell Precision 7960 Tower.
- **Processor:** Intel Xeon w7-3565X - 2.5 GHz (32 cores).
- **GPU:** NVIDIA RTX 5000 Ada Generation (32 GB GDDR6 Discrete VRAM).
- **Memory & OS:** 132GB RAM running Windows 11.

Methodology: Encoder Study



Goal: Classify text chunks as relevant or irrelevant before Q&A generation.

Irrelevant chunks include, preface, indices, and tables of contents.

Evaluated 20 encoder transformer models grouped by size

- Small: < 50M parameters
- Medium: 50-200M Parameters
- Large: > 200M Parameters

Source data came from a curated subset of Project Gutenberg documents, chunked with LangChain

Final Labelled Dataset : 6,184 Chunks and 594,711 tokens, labels were manually assigned and stored in SQLite before export for training.

Methodology: Encoder Study



Tested input lengths of 128, 256 and 512 tokens. We did not run 1024-token inputs for the encoder models because of the mismatch between the input sequence length and the model's positional embedding limit.

Training setup: HuggingFace Trainer, stratified train/test split, weighted loss for class imbalance and weighted F1 as the main selection metric as it reflects performance across both classes than accuracy alone.

Fine-tuning parameters

- Learning rates 5×10^{-5} to 1×10^{-5} .
- 2–4 epochs
- Batch size scaled with model size, from 32 to 1 for the largest models.
- 0.1 weight decay across all experiments for regularization

Initial Results: Encoder Model - Small



Model	Best Max Length	Accuracy	Weighted F1	Training Time(s)	Inference Time (s)
albert-base-v1	256	0.9854	0.9854	282.31	5.86
albert-base-v2	256	0.9806	0.9803	433.95	9.23
mobilebert	256	0.9790	0.9784	184.52	4.91
electra-small	256	0.9782	0.9779	98.24	1.96
albert-large-v2	128	0.9669	0.9656	649.74	13.77
	Average	0.9780	0.9775	329.75	7.00

Initial Results: Encoder Model - Medium



Michigan
Technological
University

Model	Best Max Length	Accuracy	Weighted F1	Training Time(s)	Inference Time (s)
deberta-v3-base	512	0.9838	0.9837	1386.09	36.84
roberta	512	0.9822	0.9822	566.19	11.96
xlnet	512	0.9790	0.9786	1223.04	30.79
deberta-v3-small	256	0.9806	0.9802	305.15	6.99
distilbert	256	0.9814	0.9812	139.70	3.23
	Average	0.9814	0.9812	724.03	17.96

Initial Results: Encoder Model - Large



Model	Best Max Length	Accuracy	Weighted F1	Training Time(s)	Inference Time (s)
deberta-large	256	0.9830	0.9827	2939.95	49.74
deberta-v3-large	256	0.9830	0.9827	3081.35	51.81
deberta-xlarge	128	0.9798	0.9793	4897.87	68.05
xlm-roberta-large	256	0.9790	0.9784	1774.17	21.24
bert-large-uncased	128	0.9782	0.9778	625.62	10.58
	Average	0.9806	0.9802	724.03	40.28

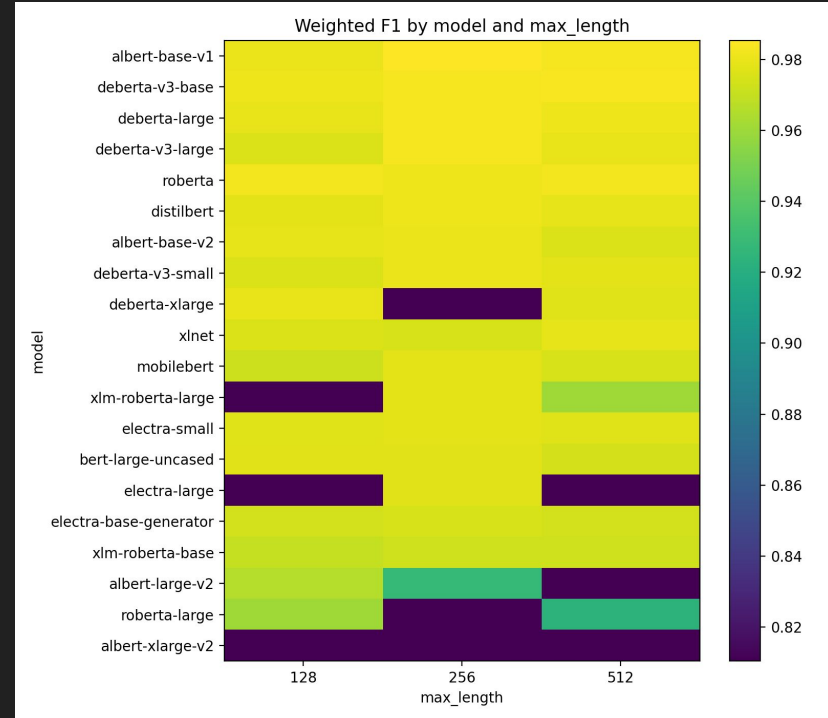
Initial Results: Encoder Model

Group	Best Model	Best Length	Accuracy	Weighted F1	Inference Time
Small	albert-base-v1	256	0.9854	0.9854	5.86s
Medium	deberta-v3-base	512	0.9838	0.9837	36.84s
Medium(Fast)	distilbert	256	0.9814	0.9812	3.23s
Large	deberta-large	256	0.9830	0.9827	49.74s

- Medium models slightly outperform large models on average, while taking less than half the average inference time.
- Small models were competitive, especially albert-base-v1, which achieved 0.9854 accuracy / 0.9854 weighted F1 at 256 tokens.
- Distilbert and deberta-v3-small were strong tradeoff models because they kept high performance with faster inference

Initial Results: Encoder Model

- Most encoder models maintained high weighted F1 across all tested input lengths.
- Performance differences by length were generally modest, but a few models showed length sensitivity and occasional sharp drops.
- The results support choosing models based on both classification quality and inference efficiency, not parameter count alone.



Methodology: Decoder Study



Models Evaluated (7B parameter range):

- Microsoft Phi-4
- Mistral-7B-Instruct-v0.3
- Qwen2.5-7B
- Google Gemma-7B

Methodology: Decoder Study



Experimental Setup:

Chunk Size	Number of Chunks	Model Max Tokens
128	1,800	128, 256, 512, 1024, 2048
256	1,800	128, 256, 512, 1024, 2048
512	1,622	128, 256, 512, 1024, 2048
1024	910	128, 256, 512, 1024, 2048

Methodology: Decoder Study



Metrics Tracked:

- Efficiency: Total elapsed time in minutes.
- Reliability: Success rate of generating valid, parseable JSON.
- Quality: Semantic accuracy (BERT F1 Score) and structural adherence (ROUGE-L).

Methodology: Decoder Study



Generated JSON Example:

```
{
  "chunk": "...Burger King sued in... federal district court... Rudzewicz contested the court's
personal jurisdiction...",
  "qa_pairs": [
    {
      "question": "What action did Burger King take?",
      "answer": "Burger King took legal action against Rudzewicz by suing in the federal
district court for the southern district of Florida."
    },
    {
      "question": "Why did Rudzewicz contest the court's personal jurisdiction over him?",
      "answer": "Rudzewicz contested the court's personal jurisdiction because..."
    }
  ]
},
...
```

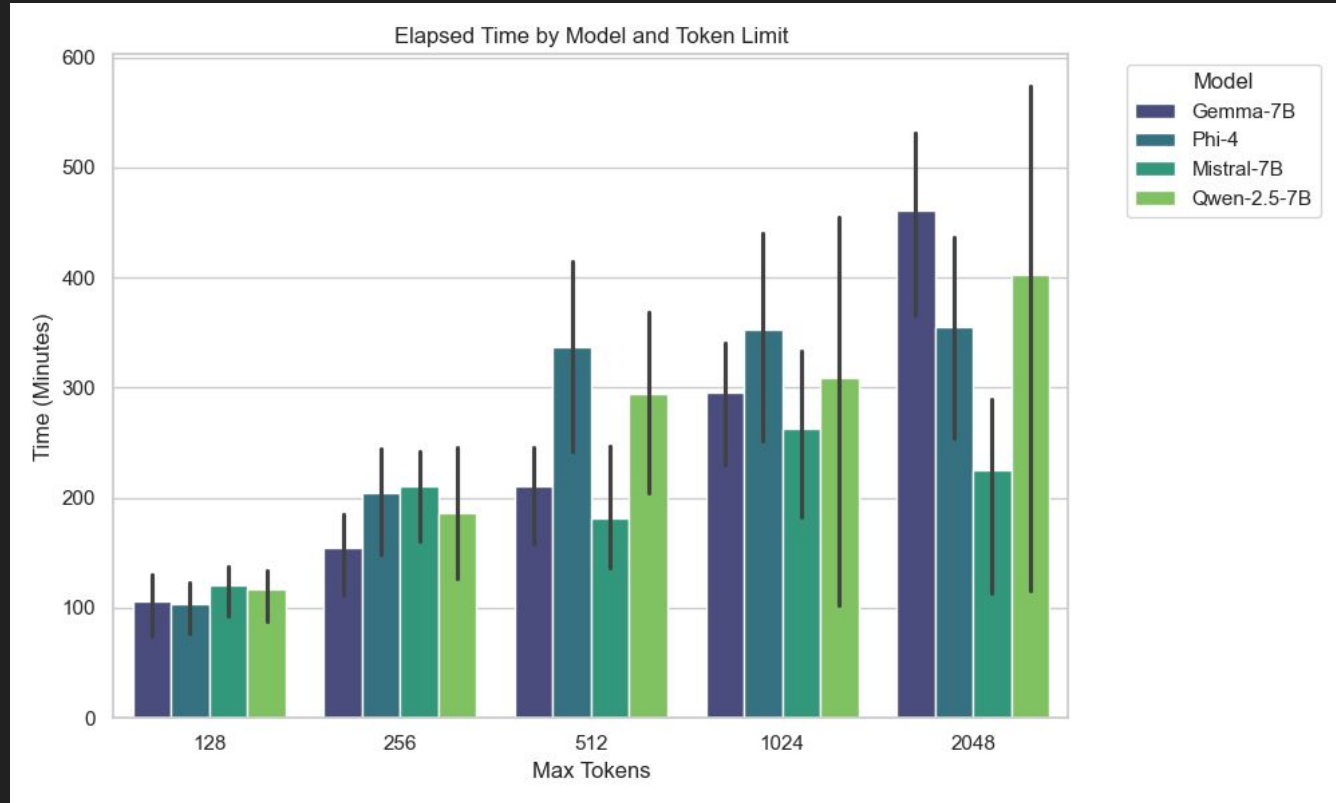
Initial Results: Decoder Model



Michigan
Technological
University

Decoder Results - Efficiency

- **Winner:** Mistral-7B.
- Consistently processed data 25-30% faster than its closest competitor (Phi-4).
- Averaged ~200 minutes per run across configurations, while others took significantly longer.



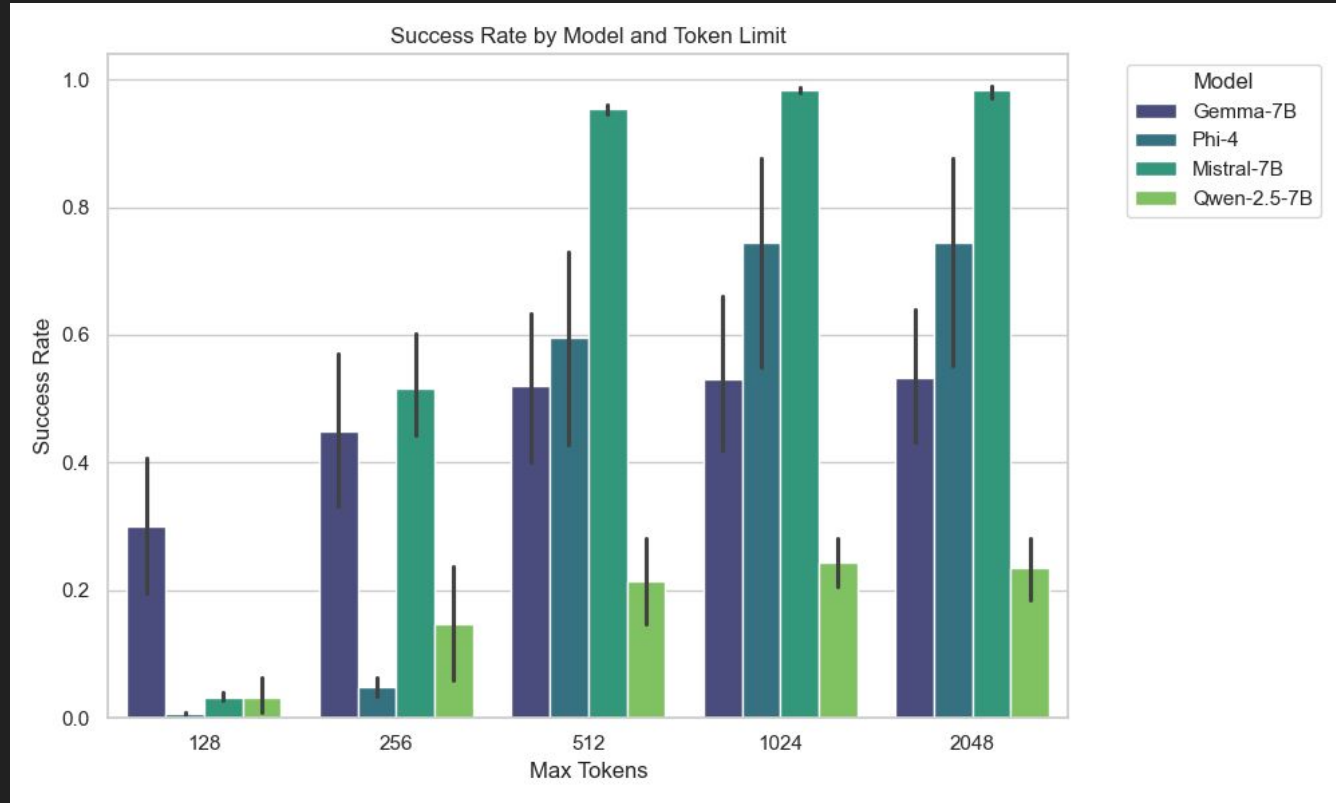
Initial Results: Decoder Model



Michigan
Technological
University

Decoder Results - Reliability

- **Winner:** Mistral-7B.
- Achieved the highest success rate for generating valid, parseable JSON outputs.
- Demonstrated the fewest critical failures, ensuring the tool remains usable for non-technical researchers.



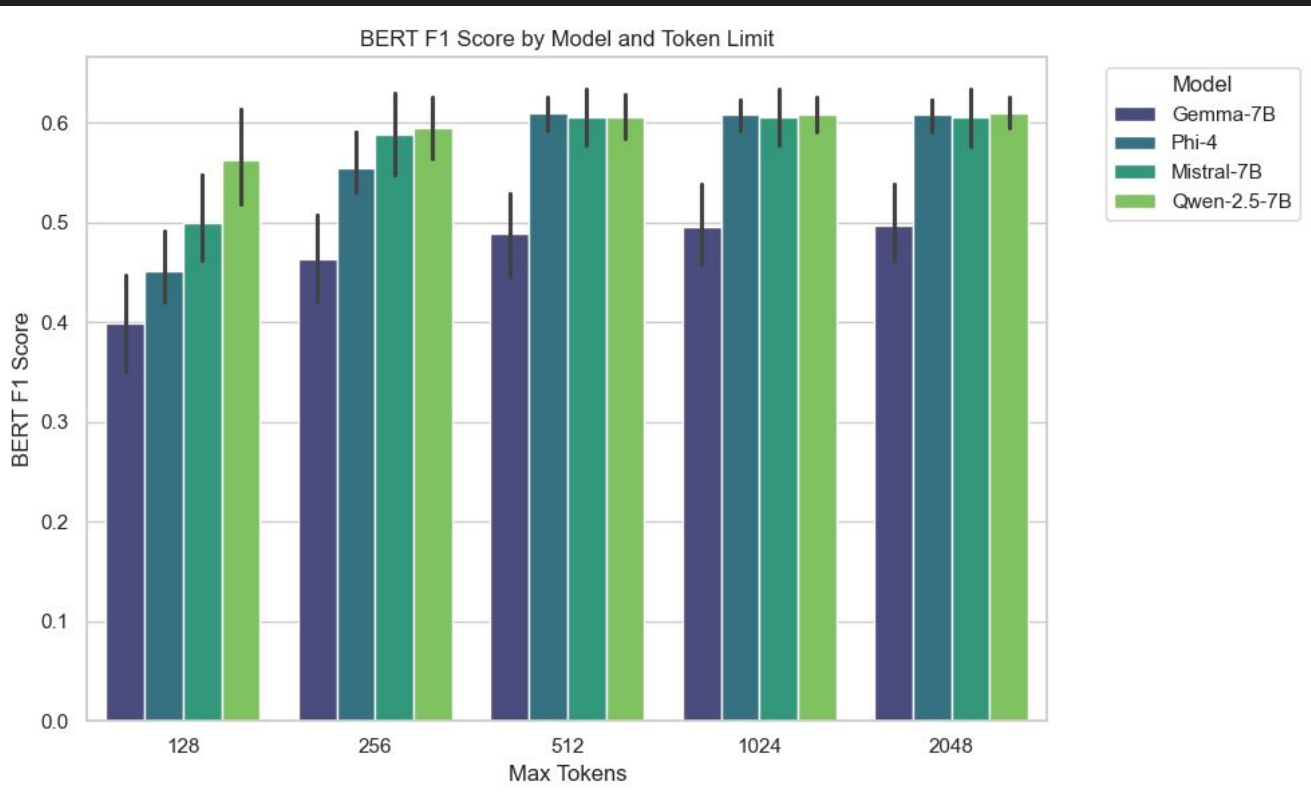
Initial Results: Decoder Model



Michigan
Technological
University

Decoder Results - Quality

- **Semantic Accuracy (BERT F1):** Mistral-7B and Qwen-2.5-7B consistently tied for the highest scores.
- Google Gemma-7B consistently scored the lowest in semantic alignment across configurations.



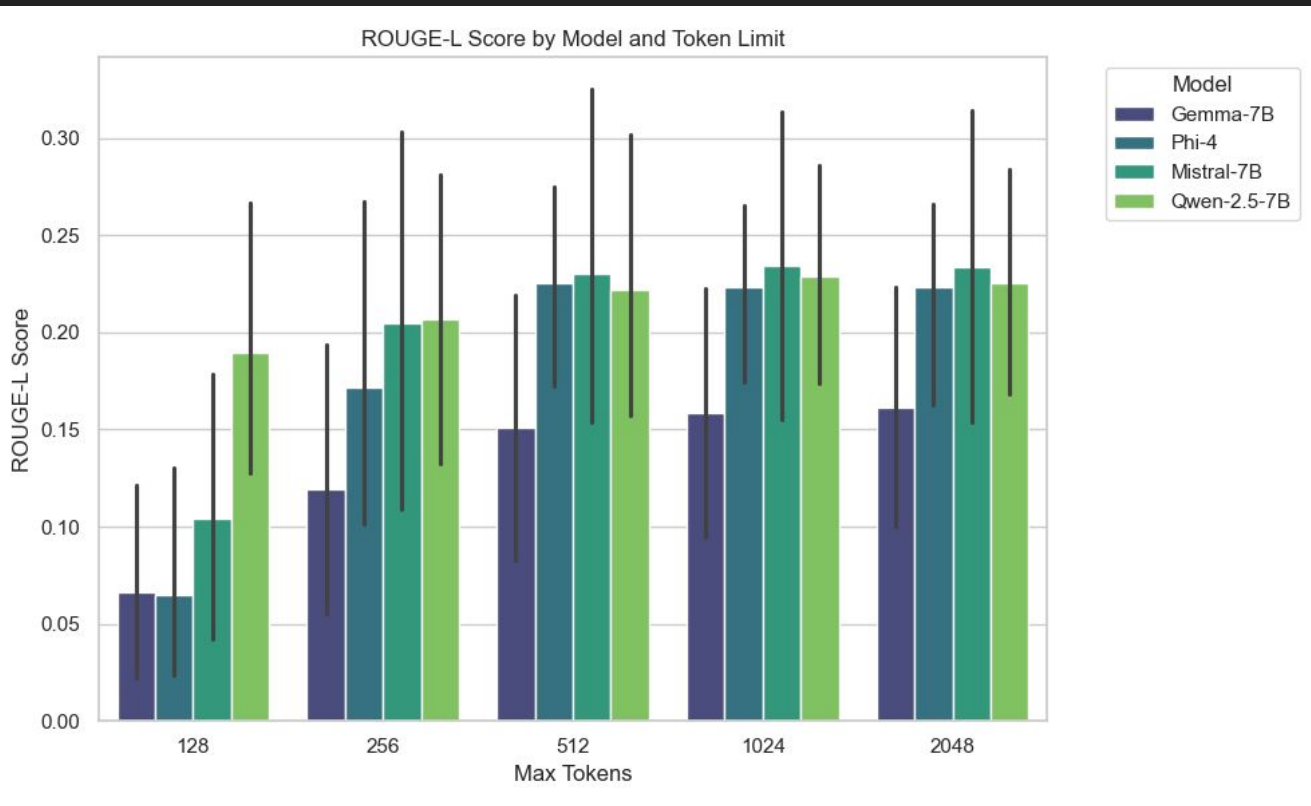
Initial Results: Decoder Model



Michigan
Technological
University

Decoder Results - Quality

- **Structural Adherence (ROUGE-L):** Mistral-7B maintained strong structural quality when outputting 512+ tokens.
- Mistral-7B and Phi-4 required a minimum output threshold of 512 tokens to effectively generate complete structures.



Conclusion: Recommended Two Models



- **Optimal Input/Output:** 256-token chunk size with a 512-token maximum output.
- **Recommended Encoder:** albert-base-v1, deberta-v3-small, or distilbert.
- **Recommended Decoder:** Mistral-7B.

Conclusion: Future Research



Michigan
Technological
University

- Conduct human studies and consult domain experts to evaluate output quality.
- Test 1024-token encoder inputs
- Validate encoder choices inside LMForge
- Run statistical significance analysis (ANOVA, MANOVA, PERMANOVA)
- Revisit training parameters and grouping choices for encoder models
- Compare LMForge's usability directly against tools like n8n and LangFlow.

Questions and Comments



Any questions or comments?